아래와 같이 설계하면 **입력 생성부터 평가 완료까지** 하나의 흐름으로 관리하기 좋습니다.

---

1. 공정가치평가 작업 (fair_value_job)

> 평가 작업(Job) 자체를 관리하는 공통 테이블

논리명	설명
공정가치평가 작업번호	Job 식별번호
평가기준일	As Of Date
상품유형	FX Forward / Bond
원거래번호	원 시스템의 Trade ID
평가상태	READY, RUNNING, SUCCESS, FAILED 등
입력폴더	Input 폴더명
출력폴더	Output 폴더명
RDP환율시각	RDP 마켓데이터 환율 시각
평가요청일시	평가 요청 시간
평가시작일시	평가 시작 시간
평가종료일시	평가 종료 시간
평가결과	SUCCESS / FAILED
오류메시지	실패 시 오류 내용
재처리횟수	재실행 횟수
생성일시	등록일
수정일시	최종 수정일
평가금액	NPV
평가통화	NPV Currency
평가소요시간	평가 수행 시간(ms 또는 초)
결과파일생성여부	Output 생성 여부

---

2. FX Forward 공정가치평가 입력정보 (fair_value_fxforward)

> portfolio.xml 생성을 위한 입력 정보

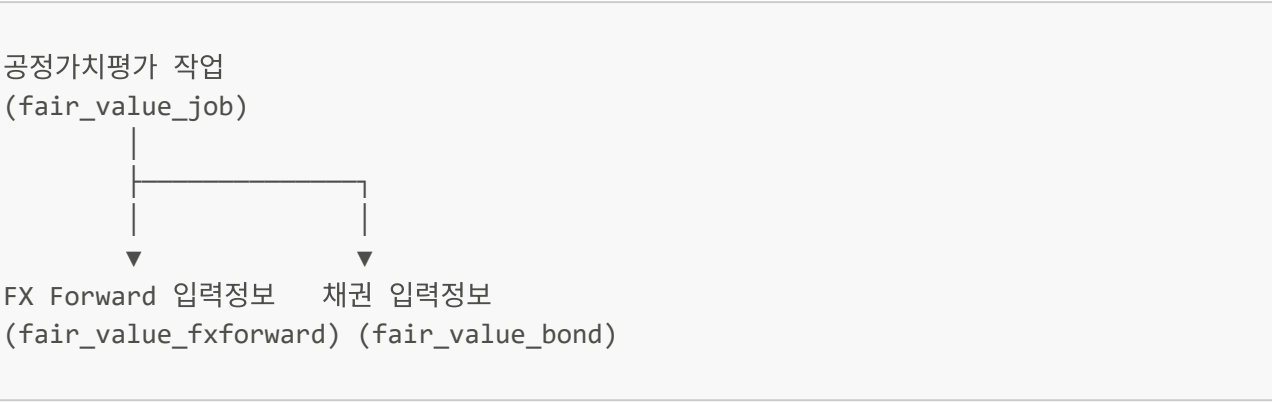
논리명	설명
공정가치평가 작업번호	Job FK
거래번호	Trade ID
거래일	Trade Date
만기일	Value Date
매수통화	Bought Currency
매수금액	Bought Amount
매도통화	Sold Currency
매도금액	Sold Amount
결제유형	Physical / Cash
거래상대방	Counterparty

3. 채권 공정가치평가 입력정보 (fair_value_bond)

> portfolio.xml 생성을 위한 입력 정보

논리명	설명
공정가치평가 작업번호	Job FK
거래번호	Trade ID
채권코드	Bond Identifier
발행기관	Issuer
통화	Currency
액면금액	Face Amount
발행일	Issue Date
만기일	Maturity Date
쿠폰금리	Coupon Rate
쿠폰주기	Coupon Frequency
일수계산방식	Day Count Convention
영업일조정규칙	Business Day Convention
결제일수	Settlement Days
거래상대방	Counterparty

관계



결과 컬럼은 어디에 저장할까?

입력 테이블(`fair_value_fxforward`, `fair_value_bond`)에는 **입력 데이터만** 저장하고, 평가 결과는 `fair_value_job`에 두는 것을 권장합니다.

예를 들어 `fair_value_job`에 아래와 같은 결과 컬럼을 추가합니다.

논리명	설명
평가금액	NPV
평가통화	NPV Currency
평가엔진	사용한 Pricing Engine
평가소요시간	평가 수행 시간(ms 또는 초)
결과파일생성여부	Output 생성 여부

이렇게 하면 **입력 정보는 상품별 테이블**, **평가 작업의 상태와 대표 결과는 공통 테이블**에 모이므로 조회와 운영이 훨씬 단순해집니다.

추가 제안

입력 XML 생성에 필요한 **시장데이터(커브, 환율, 변동성 등)는 별도 XML 파일로 공통 관리**할 가능성이 높으므로, `fair_value_job`에는 **평가기준일**, **시장데이터 버전**, **ORE 버전** 정도만 저장하는 것이 적절합니다. 이렇게 하면 동일한 시장데이터로 여러 상품을 평가했는지 추적하기도 쉬워집니다.